

Frontend Engineering Platform — Architecture & Task Roadmap

[!!IMPORTANT]

Task Selection & Submission Deliverables:

- **Deliverables Required:** Your final submission must include a **GitHub Code Repository** with your source code and a **Video Demonstration** (Loom/MP4 walkthrough) showing the working UI and interactive features.
- **Task Choice:** You can choose **at least 1 task** (Task 1, Task 2, Task 3, or Task 4) to complete and demonstrate, OR combine all 4 tasks into **1 integrated compliance workspace application**.
- **Technology Stack:** Built using **Next.js (App Router preferred)**, **TypeScript**, and **TailwindCSS**.
- **Build-From-Scratch Philosophy:** Every task is designed to be completed starting from an **empty repository** (e.g. initialized via `npx create-next-app@latest`). You generate mock data fixtures, build responsive UI components, handle local/server state, and craft polished micro-animations.

System Overview: An enterprise-grade Frontend System for the Glynac Wealth Management & Compliance Platform comprising 4 core application modules:

1. **Virtual Data Room (VDR)** (Document hierarchy, PDF/Text previewer, permission tags, & audit trails)
2. **Real-Time Compliance Chat & AI Assistant** (Streaming message interface, thread history, citations, & document attachments)
3. **Enterprise Admin Panel** (User RBAC management, rule policy editor, analytics charts, & feature flag toggles)
4. **Interactive Visual Tree Editor** (Node graph builder, drag-and-

Product Vision & Real-World User Workflow

Who Uses This Product?

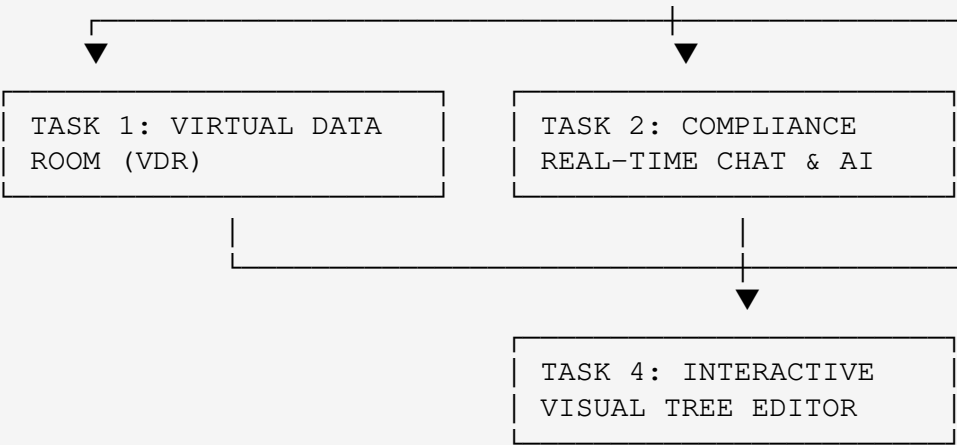
Wealth Managers, Compliance Officers, Risk Directors, and System Administrators at financial firms.

Why Do They Need It?

Financial firms handle confidential client portfolios, SEC/FINRA regulatory rules, and high-stakes compliance audits daily.

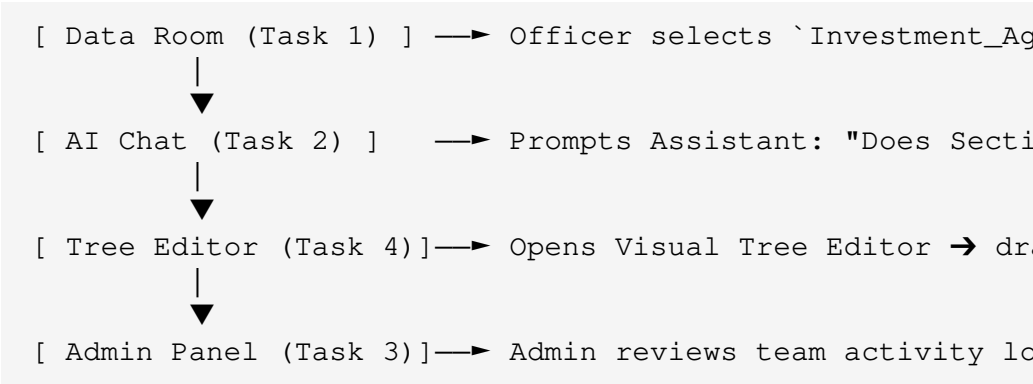
- **The Manual Problem:** Fragmented file folders, clunky legacy interfaces, and unorganized communication channels lead to missed document disclosures, slow audit response times, and compliance oversights.
 - **The Solution:** A modern, high-performance web interface built with **Next.js**, **TypeScript**, and **TailwindCSS** that unifies document management, AI chat assistance, system administration, and visual policy graph editing into a single fluid workspace.
-

System Architecture & Component Overview



🔄 End-to-End User & Integrated Workspace Flow

📄 Target Scenario: Reviewing & Updating Firm Compliance Policies



🔧 TASK 1: Build a Virtual Data Room (VDR) From Scratch (FE-1)

High-Level User Story

As a Compliance Officer & Portfolio Manager,
I want a secure, high-performance Virtual Data Room (VDR) interface,
So that I can organize, search, preview, and manage confidential financial documents with strict permission tags and full audit trail visibility.

Why Is This Needed?

Compliance teams manage thousands of sensitive client agreements, SEC disclosures, and financial audits. A clunky file browser causes lost documents and security mistakes. A VDR provides a fast, secure, and intuitive document hub with instant previews and permission controls.

📁 Concrete Task Breakdown (Build From Scratch)

1. Folder Hierarchy & File Explorer Component:

- Build a dynamic file tree & grid viewer with folder navigation, breadcrumbs, search filtering, and sorting (by name, date, file type, sensitivity level).
- Implement mock file operations: upload modal (drag-and-drop zone), folder creation, renaming, and multi-file selection.

2. In-App Document Previewer Panel:

- Build a split-screen side panel previewer supporting Text (`.txt`, `.md`), PDF/Image mock renderers, and JSON

document views.

- Add highlight markers over flagged compliance text sections in previewed documents.

3. Permission Tagging & Access Control UI:

- Implement document metadata tags (`Confidential`, `Public`, `Internal Only`, `Restricted`).
- Build a permission editor modal allowing users to assign View/Edit/Share rights to roles (Admin, Compliance Officer, Advisor, Auditor).

4. Activity Audit Log Drawer:

- Build a sliding timeline drawer showing file history events (Uploaded, Viewed, Downloaded, Flagged, Permission Changed) with timestamps and user avatars.

5. Responsive Layout & Micro-Animations:

- Ensure pixel-perfect responsiveness with TailwindCSS (dark/light mode support, crisp typography, subtle hover states, smooth transitions).
-

Scope Boundaries

- **In Scope:** Next.js pages/components, TypeScript interfaces, TailwindCSS styling, mock file system state, drag-and-drop file upload UI, search/filter, split-screen previewer, permission modal, activity log drawer.
- **Out of Scope:** Real cloud S3 bucket storage integration or serverless storage backends.

Acceptance Criteria (Definition of Done)

- ☐ **File Management:** Dynamic tree/grid explorer with breadcrumb navigation and instant search.
- ☐ **Document Previewer:** Split-screen document previewer

rendering text/PDF mocks with highlight markers.

- ☐ **Access Control:** Functional permission modal updating document security tags.
- ☐ **Audit Trail:** Activity log drawer displaying chronological file interaction history.

 Review Q&A Check:

"Show how the search filter updates the file list in real time. Demonstrate opening a file in the preview panel and viewing its audit log drawer. How are security permission tags assigned and rendered?"

🔧 TASK 2: Build a Real-Time Chat & AI Assistant App From Scratch (FE-2)

High-Level User Story

As a Compliance Advisor,

I want an interactive, real-time chat application with an integrated AI Compliance Assistant,

So that I can ask questions about regulatory rules, stream AI responses with source citations, attach documents, and manage conversation threads.

Why Is This Needed?

Navigating complex financial regulations requires quick advice and collaboration. A proper chat interface allows advisors to consult AI models, attach relevant documents, receive verifiable citations, and maintain organized chat threads.

📁 Concrete Task Breakdown (Build From Scratch)

1. Thread History Sidebar & Conversation State:

- Build a collapsible sidebar listing chat threads grouped by date (Today, Yesterday, Previous 7 Days), with thread creation, pinning, renaming, and deletion.
- Implement local/session state persistence so active conversations persist across reloads.

2. Streaming Chat Message Feed:

- Build a smooth chat message feed supporting user messages and simulated AI streaming responses (typing effect).
- Render rich Markdown content in messages (code blocks with copy buttons, formatted tables, bullet lists, bold highlights).

3. Verifiable Citation Cards & Source Link Highlights:

- Render clickable source citation pills (e.g. [FINRA Rule 2210 § 4]) within AI messages.
- Clicking a citation opens a reference popover displaying the exact excerpt and document link.

4. Document Attachment & Context Prompting:

- Build an attachment button allowing users to drag-and-drop or select files to attach to a chat prompt.
- Render attached document preview chips in the message input box before sending.

5. Message Search, Quick Prompts & User Actions:

- Add in-chat message search to jump to specific keywords.
 - Provide "Suggested Quick Prompts" chips (e.g., *"Check investment agreement for guaranteed return clauses"*).
 - Add message action buttons: Copy, Regenerate, Feedback (Thumbs Up/Down), and Edit Prompt.
-

Scope Boundaries

- **In Scope:** Next.js chat interface, TypeScript message models, TailwindCSS styling, simulated streaming text animation, Markdown renderer, citation popovers, document attachment chips, thread management, suggested quick prompts.
- **Out of Scope:** Real WebSockets backend server deployment (simulated async streaming is expected).

Acceptance Criteria (Definition of Done)

- ☐ **Streaming Feed:** Smooth typing simulation for AI responses with full Markdown and code block rendering.
- ☐ **Citations:** Clickable citation pills in messages displaying source excerpt popovers.
- ☐ **Attachments:** Functional file attachment chips in the prompt input area.
- ☐ **Thread Management:** Sidebar with thread creation, switching, pinning, and deletion.

Review Q&A Check:

*"Demonstrate sending a prompt with an attached document chip.
Show the simulated streaming AI response and click a citation pill.
How are chat threads managed and persisted?"*

🔧 TASK 3: Build an Enterprise Admin Panel From Scratch (FE-3)

High-Level User Story

As a System Administrator & Compliance Director,
I want an enterprise admin panel dashboard,
So that I can manage user roles (RBAC), configure compliance rules, monitor platform analytics, inspect system health, and toggle feature

flags.

Why Is This Needed?

Enterprise compliance software requires strict governance. Administrators need centralized controls to manage employee access, configure active SEC/FINRA rules, track review volume metrics, and control system-wide settings safely.

Concrete Task Breakdown (Build From Scratch)

1. Executive Metrics & Analytics Dashboard:

- Build an analytics dashboard featuring metric summary cards (Total Documents Reviewed, Compliance Violations Flagged, Pass Rate %, Average Review Time).
- Implement interactive charts (using Recharts, Chart.js, or custom SVG/Tailwind charts) showing review trends over time and rule violation distribution.

2. User & Role-Based Access Control (RBAC) Management:

- Build a searchable user management table with pagination, sorting, and status badges (Active, Pending, Suspended).
- Build a Role & Permission Editor modal (assigning granular permissions: Read, Write, Approve, Admin across modules).

3. Compliance Rule Policy Manager:

- Build a rule management view listing active platform rules (e.g. FINRA-2210, SEC-17a-4, DISC-09).
- Allow admins to edit rule severity (Low, Medium, High, Critical), toggle rule active status, and add custom rule definitions.

4. System Health & Activity Log Monitor:

- Build a system health monitoring panel showing API latency, database connection status, and error rate indicators.
- Include a filterable, real-time audit log stream displaying user actions across the platform.

5. Feature Flag & Configuration Toggles:

- Build a Feature Flag control panel with toggle switches (e.g., Enable AI Substring Validation, Enable Presidio PII Masking, Enable Precedent Lookup) with instant state feedback.
-

Scope Boundaries

- **In Scope:** Next.js admin layout, TypeScript data types, TailwindCSS styling, analytics charts/metrics, RBAC user table & permission modal, rule policy editor, system health monitor, feature flag toggles.
- **Out of Scope:** Connecting to live production SQL databases or cloud auth providers.

Acceptance Criteria (Definition of Done)

- ☐ **Analytics Dashboard:** Metric summary cards and interactive charts displaying review trends.
- ☐ **RBAC Management:** User table with role editing modal updating permissions in local state.
- ☐ **Policy Manager:** Rule management table supporting status toggles and severity edits.
- ☐ **Feature Flags:** Working feature flag toggle panel with active status indicators.

Review Q&A Check:

"Show the analytics metric charts and filter options. Demonstrate updating a user's role in the RBAC modal. How do feature flag toggles update system configurations?"

✂ TASK 4: Build an Interactive Visual Tree Editor From Scratch (FE-4)

High-Level User Story

As a Compliance System Architect & Workflow Designer,
I want an interactive visual tree and node graph editor,
So that I can visually build, edit, connect, and inspect hierarchical structures such as compliance rule trees, firm organization charts, or pipeline decision DAGs.

Why Is This Needed?

Complex compliance logic and organizational structures are hard to understand in flat text files. A visual, node-based tree editor empowers teams to drag-and-drop nodes, define parent-child connections, configure node properties, and export structured JSON workflows.

📁 Concrete Task Breakdown (Build From Scratch)

1. Interactive Canvas & Graph Workspace:

- Build a visual canvas workspace supporting zoom, pan, grid background toggle, and canvas reset controls (using React Flow, custom SVG/HTML5 Canvas, or drag-and-drop libraries).
- Render distinct node types (e.g., `Root Node`, `Rule Node`, `Condition Node`, `Action Node`) with color-coded badges and status indicators.

2. Drag-and-Drop Node Editing & Tree Construction:

- Build a sidebar palette allowing users to drag new nodes onto the canvas.
- Implement node connections (drawing directional edges between parent and child nodes).
- Support node operations: add child, duplicate node, delete node, and re-parent node.

3. Node Properties Inspector Side Panel:

- Build a slide-over properties panel that opens when a node is clicked.
- Allow users to edit node label, rule ID, description, severity level, assigned parameters, and metadata tags in real time.

4. Tree Search, Auto-Layout & Mini-Map:

- Implement a search bar to highlight matching nodes in the tree canvas.
- Add an "Auto-Layout Tree" button that automatically cleans up and aligns nodes into a neat hierarchical tree layout.
- Add a canvas Mini-Map navigator in the bottom corner.

5. JSON Import / Export & Validation Engine:

- Build a JSON view modal allowing users to export the current tree structure as a clean JSON object or import an external JSON file to populate the canvas.
 - Add a tree validation check that alerts the user if orphaned nodes (unconnected children) or circular references exist.
-

Scope Boundaries

- **In Scope:** Next.js page/component, TypeScript node interfaces, TailwindCSS styling, interactive canvas (zoom/pan/drag), node palette, connection edges, properties panel, node search, auto-layout, JSON import/export, tree validator.
- **Out of Scope:** Storing graph state in a remote graph database engine.

Acceptance Criteria (Definition of Done)

- ☐ **Visual Canvas:** Interactive node canvas supporting zoom, pan, grid background, and drag-and-drop node placement.
- ☐ **Node Connections:** Functional parent-child node linking and structural node editing.
- ☐ **Properties Inspector:** Slide-over panel allowing real-time node attribute modifications.
- ☐ **JSON Import/Export:** Modal exporting valid JSON tree structures and importing external tree definitions.

 Review Q&A Check:

"Demonstrate dragging a new node onto the canvas and linking it to an existing parent node. Edit node properties in the inspector panel. Show the JSON export output and tree validation check."

6. Reference

6.1 Glossary

Term	What It Means Here
App Router	Next.js 13/14+ directory-based routing architecture (app/ directory).
TypeScript Strict	Ensuring all component props, state objects, and API models are strongly typed with interfaces/ types.
TailwindCSS	Utility-first CSS framework for layout, typography, colors, and responsive design.
Virtual Data Room (VDR)	Secure online repository for document storage, distribution, and compliance previewing.

Citations	References embedded in AI chat messages linking directly to source rule passages or document excerpts.
RBAC	Role-Based Access Control — restricting system features based on assigned user roles.
Feature Flags	Toggle switches enabling or disabling system capabilities without code deployments.
Tree Editor	Visual canvas tool for building node-based graph hierarchies, workflows, or organizational trees.
Auto-Layout	Algorithmically positioning tree graph nodes into clean hierarchical rows and columns.

6.2 When You Are Stuck

- 🕒 **Ask after 4 hours:** If stuck on canvas drag-and-drop math or Tailwind layout bugs for more than half a day, reach out to your mentor.
- 🧠 **Prioritize Visual Excellence:** Use clean color palettes (dark/light mode contrast), smooth micro-interactions (cubic-bezier transitions), loading skeletons, and polished typography.
- 📦 **Component Modularity:** Keep components small, reusable, and cleanly typed (`components/vdr/`, `components/chat/`, `components/admin/`, `components/tree/`).
- 📝 **Mock State Quality:** Create rich, realistic TypeScript mock datasets (`mockDocs.ts`, `mockChats.ts`, `mockUsers.ts`, `mockTree.ts`) so the UI feels alive.

6.3 What Good Looks Like at the End of the Month

1. You can demonstrate a fluid, highly responsive Next.js application built with TypeScript and TailwindCSS.
2. Your components handle edge cases (empty states, loading skeletons, modal toggles, search filtering).
3. You recorded a clear video demonstration showing your working task(s) or integrated application.

4. You can clearly explain your component architecture, state management choices, and UI design decisions.

None of that requires your code to connect to live production databases. It requires you to have built a state-of-the-art frontend experience that feels production-ready.